



Base de datos y backend

Guía paso a paso

MySQL · Node.js · Express

Plataforma de Contact Center

BPM Consulting

Plataforma de Contact Center · BPM Consulting

Este es el paso que sigue después de la telefonía. Al terminar, la plataforma va a guardar los usuarios, contactos, llamadas y tipificaciones en MySQL, en lugar de tenerlos escritos en el código.

Sigue los pasos en orden. Cada uno se verifica antes de pasar al siguiente, para que no avances sobre algo roto.

Qué vas a montar

```
NAVEGADOR —————> BACKEND —————> MySQL
(la plataforma)      (Node.js)      (los datos)
```

El navegador nunca habla con MySQL. Si lo hiciera, cualquiera podría abrir las herramientas del desarrollador, ver la contraseña de la base y borrarla entera.

El backend es el intermediario: recibe la petición, comprueba quién la hace y si tiene permiso, consulta la base y devuelve solo lo que corresponde.

PASO 1 · Instalar MySQL

Ya lo tienes instalado (MySQL 8.0.41). Solo hay que dejar el comando disponible de forma permanente:

```
[Environment]::SetEnvironmentVariable("Path", $env:Path + ";C:\Program Files\MySQL\MySQL Server 8.0\bin", "User")
```

Cierra VS Code y vuelve a abrirlo. Después:

```
mysql --version
```

Verificación

Debe responder con la versión. Si dice que no reconoce el comando, la ruta es otra; búscala así:

```
Get-ChildItem "C:\Program Files" -Recurse -Filter "mysql.exe" -ErrorAction SilentlyContinue |  
Select-Object -First 3 FullName
```

PASO 2 · Crear la base de datos

En la carpeta del backend:

```
Get-Content esquema.sql | mysql -u root -p
```

Te pide la contraseña de `root`, la que pusiste al instalar MySQL.

PowerShell no admite el símbolo `<`. Por eso se usa `Get-Content` en vez de `mysql -u root -p < esquema.sql`, que es lo habitual en Linux.

Qué hace ese archivo

Crea la base `bpm_contact` con 18 tablas y algunos datos para empezar: cinco usuarios, cuatro campañas, cinco contactos y el catálogo de tipificación con las categorías que definió la operación.

Verificación

```
mysql -u root -p -e "USE bpm_contact; SHOW TABLES;"
```

Deben aparecer 18 tablas.

```
mysql -u root -p -e "USE bpm_contact; SELECT usuario, extension FROM usuario;"
```

Debe mostrar `ana` con extensión `1001` y `pedro` con `1002`, que son las que existen en tu Asterisk.

Cuidado: el archivo empieza con `DROP DATABASE IF EXISTS`, que borra todo si ya existía. Está bien mientras aprendes; **nunca lo ejecutes cuando ya haya datos reales.**

PASO 3 · Crear el usuario de la aplicación

El backend **no debe usar** `root`. Si tiene un fallo, con `root` podría borrar tablas enteras.

```
mysql -u root -p -e "CREATE USER 'bpm_app'@'localhost' IDENTIFIED BY 'BpmApp2026'; GRANT SELECT, INSERT, UPDATE, DELETE ON bpm_contact.* TO 'bpm_app'@'localhost'; FLUSH PRIVILEGES;"
```

Ese usuario puede leer y escribir datos, pero **no puede crear ni borrar tablas**. Limita el daño posible.

Verificación

```
mysql -u bpm_app -pBpmApp2026 -e "USE bpm_contact; SELECT COUNT(*) FROM usuario;"
```

Debe responder 5. Si dice acceso denegado, la contraseña no coincide.

PASO 4 · Configurar el backend

```
cd C:\dev\backend
npm install
copy .env.ejemplo .env
```

Abre `.env` y déjalo así:

```
PUERTO=3001

BD_HOST=localhost
BD_PUERTO=3306
BD_USUARIO=bpm_app
BD_CLAVE=BpmApp2026
BD_BASE=bpm_contact

JWT_SECRETO=cambia-esto-por-una-cadena-larga-y-aleatoria
JWT_HORAS=8

PBX_WSS=wss://192.168.114.18:8089/ws
PBX_DOMINIO=192.168.114.18
PBX_ICE=stun:stun.l.google.com:19302
PBX_CLAVE_FIJA=ClaveWebRTC123!

CORS_ORIGENES=http://localhost:8080
```

Qué significa cada bloque

BD_* — cómo se conecta a MySQL. Debe coincidir con el usuario que creaste en el paso 3.

JWT_SECRETO — la cadena que firma los tokens de sesión. Si alguien la conoce, puede fabricar sesiones falsas. En el servidor génerala con `openssl rand -base64 48`.

PBX_* — los datos que el backend le entrega al navegador junto con la credencial. Son los mismos que hoy tienes en `js/config.js`.

PBX_CLAVE_FIJA — provisional. Mientras Asterisk no lea su configuración desde MySQL, la clave SIP no puede generarse por sesión. Cuando eso exista, se deja vacía y el backend genera una temporal.

CORS_ORIGENES — desde qué direcciones acepta peticiones. Si la plataforma corre en otro puerto, hay que agregarlo aquí o el navegador bloquea las llamadas.

El archivo `.env` **nunca se sube al repositorio**: tiene contraseñas. Por eso existe `.env.ejemplo`, que es la misma estructura sin valores.

PASO 5 · Arrancar el backend

```
npm start
```

Verificación

Debe mostrar:

```
BPM Consulting – Backend
```

```
Base de datos      8.0.41-MySQL  
                  bpm_app@localhost/bpm_contact  
Central           wss://192.168.114.18:8089/ws  
Escuchando en     http://localhost:3001
```

Si dice «**No se pudo conectar a la base de datos**», revisa las variables `BD_*`. El mensaje de abajo te dice la causa exacta.

Deja esa terminal abierta. El backend tiene que estar corriendo.

PASO 6 · Comprobar que funciona

En **otra** terminal:

```
cd C:\dev\backend  
node probar.js
```

Ejecuta 40 pruebas contra la base real: autenticación, permisos por rol, creación de usuarios, cambio de rol, resistencia a inyección SQL y las rutas de operación.

Verificación

Deben pasar las 40. Si alguna falla, el mensaje dice cuál y por qué.

PASO 7 · Poner las contraseñas reales

El esquema trae un hash de ejemplo. Para poder entrar, genera uno de verdad:

```
node -e "console.log(require('bcryptjs').hashSync('demo1234',12))"
```

Copia el resultado y aplícalo:

```
mysql -u root -p -e "USE bpm_contact; UPDATE usuario SET clave_hash='EL-HASH-QUE-COPIASTE';"
```

Con eso, todos los usuarios entran con la contraseña `demo1234`.

Por qué se guarda así

Nunca se guarda la contraseña. Se guarda el resultado de aplicarle **bcrypt**, que es irreversible:

```
demo1234 → $2b$12$N9qo8uL0ickgx2ZMRZoMye...
```

Al iniciar sesión se aplica lo mismo a lo que escribió el usuario y se comparan los resultados. Si alguien roba la base, no obtiene las contraseñas.

PASO 8 · Conectar la plataforma

En `js/config.js` de la plataforma:

```
api: 'http://localhost:3001/api',
```

Guarda y recarga con **Ctrl+Shift+R**.

Verificación

Entra con `ana` y `demo1234`. En la traza debe decir:

```
Backend conectado: http://localhost:3001/api
```

Si dice «Sin backend: datos locales», la dirección no se guardó o está mal escrita.

Qué cambia a partir de aquí

Antes	Ahora
Usuarios en <code>js/servicio.js</code>	En la tabla <code>usuario</code> de MySQL
Contraseña sin validar	Validada con bcrypt
Contactos escritos a mano	Consulta a la tabla <code>contacto</code>
Catálogo fijo en el código	Según la campaña del agente
Credencial SIP del <code>config.js</code>	Emitida por el servidor, con vencimiento
Tipificaciones que se perdían	Guardadas en la base

La plataforma se ve igual. Lo que cambia es de dónde salen los datos.

PASO 9 · Volver atrás si algo falla

Si el backend se cae o hay que presentar sin él, deja la dirección vacía:

```
api: '',
```

La plataforma vuelve a los datos locales y sigue funcionando. Eso es deliberado: nunca dependes de que el backend esté arriba para poder mostrar la interfaz.

Cómo está hecho el backend

```

backend/
├── .env           claves y direcciones (NO se sube)
├── esquema.sql   crea la base y sus tablas
├── probar.js     40 pruebas contra la base
├── probar-integracion.mjs 32 pruebas de la plataforma contra el backend
└── src/
    ├── servidor.js  arranque y montaje de rutas
    ├── config.js    lee el .env en un solo sitio
    ├── bd.js        conexión con MySQL
    ├── auth.js      contraseñas, tokens y permisos
    ├── asterisk.js  creación de extensiones (pendiente)
    └── rutas/
        ├── sesion.js  iniciar sesión y credencial SIP
        ├── usuarios.js crear, editar y desactivar
        └── operacion.js campañas, contactos, tipificación, pausas
  
```

Las tres reglas del código

Consultas siempre con parámetros. Nunca se pega un valor dentro del texto de la consulta:

```

// MAL – así te borran la base
`SELECT * FROM usuario WHERE usuario = '${nombre}`

// BIEN
bd.una('SELECT * FROM usuario WHERE usuario = ?', [nombre])
  
```

Si alguien escribe como usuario `' OR '1'='1'`, la primera forma lo ejecuta como código. La segunda lo trata como texto. Se llama inyección SQL y está probado en la batería.

Los permisos se validan en el servidor. Ocultar botones en el navegador mejora la experiencia, pero no es seguridad: cualquiera puede enviar la petición a mano. Por eso cada ruta lleva su guarda:

```
router.post('/', auth.exigir('usuarios'), ...)
```

Las operaciones que tocan dos sitios van en transacción. O se hacen las dos, o ninguna. Si no, quedarían usuarios sin extensión.

Las tablas, por grupos

Personas y acceso

`rol`, `permiso`, `rol_permiso`, `usuario`, `usuario_campana`, `sesion`

`rol_permiso` es la que hace que cambiar el rol de una persona cambie lo que ve, sin tocar código.

`sesion` guarda las credenciales SIP temporales, con su vencimiento.

Operación

`campana`, `pausa_tipo`, `pausa`

En `campana` está el campo más importante del esquema:

```
cola_asterisk VARCHAR(80) UNIQUE
```

Es el **punte con Asterisk**. Guarda el nombre exacto de la cola. De ahí sale qué catálogo de tipificación ve el agente y en qué panel de supervisión aparece la llamada.

Contactos

`contacto`

Con los campos de la ficha: tipo y número de documento, teléfono secundario, correo y descripción del requerimiento.

El índice por teléfono es **crítico**: se consulta en cada llamada entrante y tiene que responder antes de que el agente conteste.

Telefonía

`interaccion`, `tramo`, `tipificacion`

Aquí está la decisión que más se equivoca. Una llamada transferida genera **varios canales** en Asterisk. Si guardas cada canal como una llamada, los reportes salen inflados y el historial del cliente se ve partido.

La solución es el campo:

```
linkedid VARCHAR(64) NOT NULL UNIQUE
```

Asterisk asigna el mismo `linkedid` a todos los canales de una misma llamada, incluidas las patas de una transferencia. Al ser único, agrupa los tramos.

Ejemplo probado: entra una llamada, la atiende Ana 172 segundos y la transfiere a Sandra, que habla 206 más. Resultado: **una** interacción de 390 segundos con **dos** tramos.

Formularios

`formulario`, `formulario_campo`, `formulario_respuesta`, `formulario_valor`

En `formulario_respuesta` hay dos fechas: `creada` (cuándo la llenó el agente) y `recibida` (cuándo llegó al servidor). Pueden diferir mucho si el agente la llenó sin conexión.

Auditoría

`auditoria` — quién hizo qué y cuándo.

Consultas para practicar

Entra a la base:

```
mysql -u root -p bpm_contact
```

Ver una tabla:

```
SELECT * FROM usuario;
```

Unir dos tablas. Los usuarios guardan `campana_id`, un número; el `JOIN` va a buscar a qué campaña corresponde:

```
SELECT u.nombre, u.extension, c.nombre AS campana
FROM usuario u
JOIN campana c ON c.id = u.campana_id;
```

El reporte de productividad, que es lo que hoy la plataforma genera con datos inventados:

```
SELECT u.nombre AS agente,
       COUNT(t.id) AS llamadas,
       SUM(t.segundos) AS segundos_hablados,
       ROUND(AVG(t.segundos)) AS promedio
FROM tramo t
JOIN usuario u ON u.id = t.usuario_id
WHERE DATE(t.inicio) = CURDATE()
GROUP BY u.id
ORDER BY llamadas DESC;
```

Para salir: `exit`

Respaldos

Antes de cualquier cambio grande:

```
mysqldump -u root -p bpm_contact > respaldo_2026-09-09.sql
```

Para restaurar:

```
Get-Content respaldo_2026-09-09.sql | mysql -u root -p bpm_contact
```

En producción, programado a diario.

Lo que falta después de esto

Registro automático de llamadas. Hoy la tipificación se guarda, pero la llamada en sí no. Falta el proceso que escucha el **AMI** de Asterisk y crea las filas de `interaccion` y `tramo`, agrupando por `linkedid`.

Eventos en vivo. El panel de supervisión muestra datos de ejemplo. Para que sea real necesita esos mismos eventos del AMI reenviados al navegador por WebSocket.

Credenciales SIP por sesión. Hoy se usa la clave fija del `.env`. Para que sean temporales de verdad, Asterisk tiene que leer su configuración desde MySQL.

Desplegar en el servidor. Instalar Node y MySQL en `192.168.114.18`, y poner el backend detrás de Apache para que la plataforma lo llame como `/api`.

Anexo · Verificación de esta guía

Todo lo descrito fue ejecutado contra MariaDB 10.11 real:

- El esquema crea 18 tablas sin errores
- El backend arranca y se conecta
- **40 pruebas** del backend: autenticación, permisos, usuarios, inyección SQL
- **32 pruebas de integración:** el `servicio.js` real de la plataforma contra el backend real, verificando login, credencial SIP, ficha del contacto con todos sus campos, catálogo de tipificación con las cinco categorías, pausas y cierre de sesión